Assignment 1a

Title: Web Page Design

Problem Statement: Create a responsive web page which shows the ecommerce/ college/ exam admin dashboard with sidebar and statistics in cards using HTML, CSS and Bootstrap.

Objective: Apply HTML, CSS and Bootstrap classes and demonstrate a web page that is responsive.

S/W Packages and H/W apparatus used:

Linux OS: Ubuntu/Windows, VSCode editor.

PC with the configuration as Pentium IV 1.7 GHz. 128M.B RAM, 40 G.B HDD, 15''Color Monitor,

Keyboard, Mouse.

References:

1. Steven M. Schafer, "HTML, XHTML and CSS", Wiley India Edition, Fourth Edition,978- 81-265-1635-3

Theory:

1. HTML

- HTML stands for Hypertext Markup Language. HTML is rendered on a Web Browser.

HTML has a set of elements that describes the structure of a Web page.

a. HTML Elements

- The HTML **element** comprises of a start tag, the content and the end tag:
- <tagname>Content</tagname>

- The main HTML element is <html> … </html>. All the HTML will be enclosed within it.

b. HTML Boilerplate

<!DOCTYPE html>

)

```
<html lang="en">
  <head>
      <meta charset="UTF-8">
      <meta name="viewport" content="width=device-width, initial-scale=1.0">
      <title>HTML 5 Boilerplate</title>
      <link rel="stylesheet" href="style.css">
  </head>
  <body>
     Other HTML elements are per choice of User
     <script src="index.js"></script>
  </body>
</html>
```

- HTML Boilerplate is the HTML Standard structure using which the user can create Web Pages.
- <head> and <body> are the two main tags. <head> is where the meta information, title of Web Page and CSS files can be embedded. <body> holds the user's code and any JavaScript files can also come below.

c. Block-Level vs. Inline Elements

- **The Block-level elements** occupy the entire width of the Web Page (height is automatic as per the content within element, if not explicitly mentioned).
- Common Block-Level Elements are , <section>, <form>, <main>, <table>, <h1> to <h6>, <nav>, <header>, <p>, <hr>, <ol>, <ul>, <li> etc.
- **The inline elements** take up the width on the browser as much its own width only.

- Inline Elements can sit next each other in a single row.
- Inline Elements can also be nested within the Block-level elements
- Common Inline Elements are <a>, <b>, <i>, <img>, , <input>, <label>, <button> etc.

d. HTML Attributes

- An attribute is a key-value pair written within an element.
- An attribute is used to add more specific details to an element.
- For e.g. You may use a "style" attribute in element to change the color of text or add a background …. and much more

- **Case Study: The <a> tag**
- The <a> or anchor tag is used to add a hyperlink.
- In this slide we shall see the different attributes associated with <a> tag.

1. **href** – used to mention the link
   i. e.g. CLICK ME
2. **target** – indicates where to open the linked document.
   i. e.g. CLICK ME

The web page is opened in same tab ii. e.g. CLICK ME

The web page is opened in a new window or tab

3. **style** – used to add styles
   1. e.g. CLICK ME

2. **Cascading Style Sheets (CSS)**

- CSS, also known as Cascading Style Sheets is used to style or achieve the desired look and feel of the web page.
- CSS has a wide range of properties that can facilitate the user from changing the color of text to applying animations.

a. CSS Syntax

selector{ property:value; property:value; }

Selector could be element selector or id selector or class selector or universal selector etc..
A property may have various values.

e.g. if property is "font-style"

It can have 2 values: normal, italic

b. Types of CSS

- **Inline CSS** – It is written within the HTML element itself using the style attribute.

  e.g. I am a div element

- **Internal CSS** – It is written within the HTML document using <style> tag.
- **External CSS** – It is written in a separate CSS file. The CSS can then be linked to the

  HTML file. CSS file is saved with .css extension

c. CSS Selectors

- CSS Selectors are the various ways for writing a CSS.
- The common selectors are:

1. Element Selector

The element selector uses the name of HTML tag. e.g.

HTML

```
<div>I am div 1</div>
<div>I am div 2</div>
```

CSS

```
div
{
width: 100px; border:
1px solid grey;
}
```

2. ID Selector

The id selector uses the unique id of HTML tag to apply CSS. e.g.

HTML

```
<div id="div1">I am div 1</div>
```

CSS

```
#div1 { width: 100px;
border: 1px dashed orange;

}
```

3. Class Selector

The class selector uses the class of HTML tag to apply CSS.

e.g. **HTML**

```
<div class="mydiv">I am div 1</div>
```

CSS

```
.mydiv{ width: 50%; border: 1px
dashed orange;}
```

4. Universal Selector

The universal selector applies CSS to each HTML element of the Web Page. Common styling like font-family, page width can be specified using universal selector

e.g. **CSS** *{

font-family: sans-serif;

font-weight: 700; text-

align: justify;

}

5. Grouping Selector

In grouping selector all the elements which require same style can be grouped together and a common set CSS rules can be applied to them.

e.g. **HTML**

```
<div>I am a div</div>
<p>I am paragraph 1</p>
<p id="p1">I am paragraph 2</p>
```

CSS

```
div, #p1 {
text-align: center; color:
red;
}
```

6. Combination Selector

When there is a relationship between HTML elements, the combination selector can be used to apply rules.

There are four different combination selectors in CSS:

- descendant selector (space)

- child selector (>)
- adjacent sibling selector (+)
- general sibling selector (~)
-

7. Pseudo-class Selector

A pseudo-class is used to define a specific state of an HTML element.

:link, :hover

```
<p><a href="#" target="_blank">Click ME</a></p>
e.g.  a:link
{   color:
red; }
a:hover {
color:
hotpink;

}
```

:checked

```
    <form action="#">
<input type="radio" value="male" name="gender"> Male<br>
<input type="radio" value="female" name="gender"> Female<br> </form> input:checked {  outline: 2px solid deeppink;

}
```

3. Bootstrap

- Bootstrap is a light-weight library of CSS.
- Bootstrap 5 is common used version as of now.
- It has a wide range of class that works perfectly well on all browsers.

- These classes have in-built CSS associated with them.
- e.g. class="card" automatically applies the following CSS:

position: relative; display:

flex; flex-direction:

column; min-width: 0;

a. Bootstrap Grid System

- Bootstrap Grid allows 12 columns in a row on the web page.
- The Bootstrap 5 grid system has six classes:
  - .col- (extra small devices - screen width less than 576px) ○ .col-sm- (small devices - screen width equal to or greater than 576px) ○ .col-md- (medium devices - screen width equal to or greater than 768px) ○ .col-lg- (large devices - screen width equal to or greater than 992px) ○ .col-xl- (xlarge devices - screen width equal to or greater than 1200px) ○ .col-xxl- (xxlarge devices - screen width equal to or greater than 1400px)
-

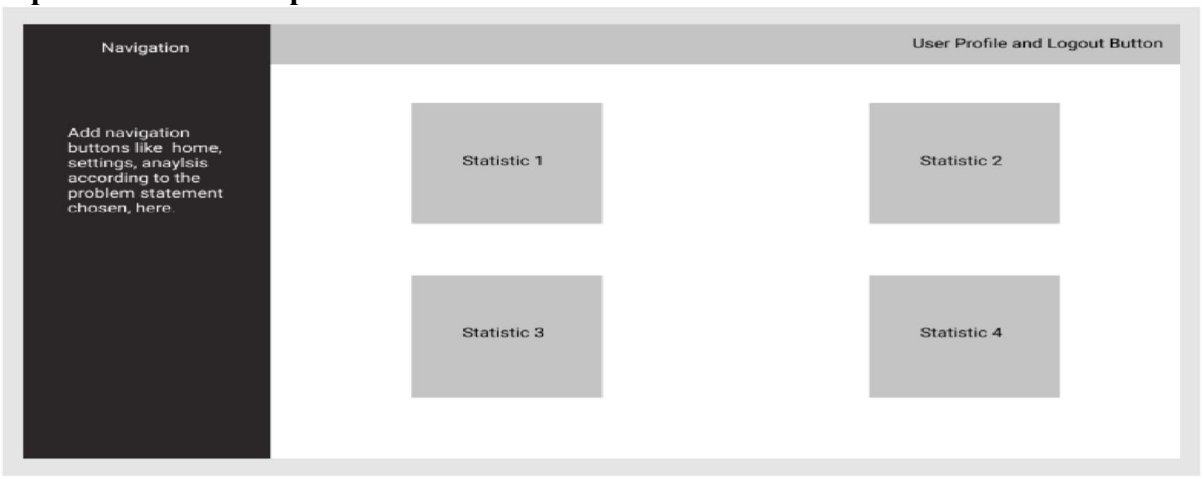
- The first * may hold one of the values: xs, sm, md, lg, xl, xxl
- The second * may hold a number (the summation of all numbers in a row must be less than or equal to 12)

b. How to add Bootstrap CDN links to your code

```
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0alpha1/dist/css/bootstrap.min.css"
rel="stylesheet">
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0alpha1/dist/js/bootstrap.bundle.min.js"></script>
 </head>
```

Source: getbootstrap.com

Implementation: Sample Dashboard



Sample HTML tags to be used

1. p
2. body
3. All table tags
4. All heading tags
5. a
6. div
7. title
8. head
9. li
10. ol
11. ul

12. html
13. br
14. hr
15. img
16. link
17. header etc…

Sample Bootstrap classes to be used

1. container
2. container-fluid
3. card
4. card-body
5. card-title
6. card-text
7. btn
8. btn-primary (with variations) etc…

Conclusion: Thus, we have applied HTML, CSS and Bootstrap classes and demonstrated a web page that is responsive.

Assignment 1b

Title: JavaScript AJAX Implementation

Problem Statement: Write a JavaScript Program to get the user registration data and push to array/local storage with AJAX POST method and data list in new page.

Objective: To understand the working of JS, AJAX, XmlHttpRequest by accepting the User registration data.

S/W Packages and H/W apparatus used:

Linux OS: Ubuntu/Windows, VSCode editor.

PC with the configuration as Pentium IV 1.7 GHz. 128M.B RAM, 40 G.B HDD, 15''Color Monitor,

Keyboard, Mouse.

References:

1. Ivan Bayross,"Web Enabled Commercial Application Development Using HTML, JavaScript, DHTML and PHP, BPB Publications,4th Edition,ISBN:978-8183330084.

Theory:

1. **HTML Form**

   An HTML form is collection of

   - Textbox (to fill info like Name, Roll No etc)
   - Textarea (to fill info like Address or lengthy text)
   - Select (to select value/s from a dropdown list)
   - Radio Button (to make one selection like Yes/No, Male/Female)
   - Check Box (to select one or more options from limited choices)

- Submit Button (to submit the filled form)

..and many more elements all contributing to the collection of info from user.

The **<form> tag** is the starting point for any HTML form The
attributes used within <form> are:

1. action: <form action="welcome.html">

After the form is submitted, filename/URL mentioned in “action” shall hold the output/processing.

2. method: <form action="welcome.html" method= "get/post">

The data could be sent as URL variables (using method="get") or as HTTP post transaction (using method="post").

2. JavaScript

JavaScript is a front-end scripting language.

JavaScript can be added to the HTML file in two ways:

- Internal JavaScript
- External JavaScript

Internal JavaScript: JS code is directly added to the HTML file using the <script> & </script> tags. The <script> tag can either be placed inside the <head> or the <body> tag according to the programming requirement.

External JavaScript: A separate file with a **.js** extension is prepared, where the only the JavaScript code is written. This JS file can be embedded in HTML file by using

<script src=”file_name.js”> tag, and this <script> may reside inside the <head> or <body> tag of the HTML file.

JavaScript Used to:

It is primarily used to develop websites and web-based applications. JavaScript is a front-end scripting language.

- **Create Interactive/Dynamic Websites:** JavaScript is used to make web pages dynamic and interactive. Using JavaScript the DOM can be altered on the fly based on the inputs or activities of the user.
- **Building Applications:** JavaScript can be employed to make web and mobile applications. To build web and mobile apps, the most popular JavaScript frameworks like – ReactJS, React Native, Node.js are used.
- **Web Servers:** A robust server applications using JavaScript can be created. To be precise the JavaScript frameworks like Node.js and Express.js are used to build these servers.

3. XmlHttpRequest

XMLHTTPRequest object is an API used to fetch data from the server. XMLHTTPRequest is used in Ajax programming. It can retrieve any format of data such as JSON, XML, text, etc. It requests for data in the background without the page refresh and loads response on the page. An object of XMLHTTPRequest is used for asynchronous communication between client and server.

Properties of XMLHttpRequest Object:

- **onload:** When the request is served and the response is ready, it defines a function to be called.
- **onreadystatechange**: A function is called whenever the readyState property changes.
- **readyState**: It holds the current status of the XMLHttpRequest. There are five states of a request:

- **readyState= 0**: It represents the Request not initialized.
- **readyState= 1**: Establishment of server connection.
- **readyState= 2**: Request has been received
- **readyState= 3**: During the time of processing the request
- **readyState= 4**: Response is ready after finishing the request

- **responseText**: It returns the response data received by the request in the form of a string.
- **responseXML**: It returns the response data received by the request in XML format.
- **status**: It returns the status number of the request. (i.e. 200 and 404 for OK and NOT FOUND respectively).
- **statusText**: It returns the status text in form of a string. (i.e. OK and NOT FOUND for 200 and 404 respectively).

4. Ajax

AJAX is abbreviation for Asynchronous JavaScript and XML. AJAX also hits the request to the server and loads the data on the HTML without the page refresh.

Examples of applications using AJAX: Gmail, Google Maps, Youtube, and Facebook tabs.

With the jQuery AJAX methods, one can request text, HTML, XML, or JSON from a remote server using HTTP Get and/or HTTP Post

The parameters specify one or more name/value pairs for the AJAX request.

Possible names/values in the table below:

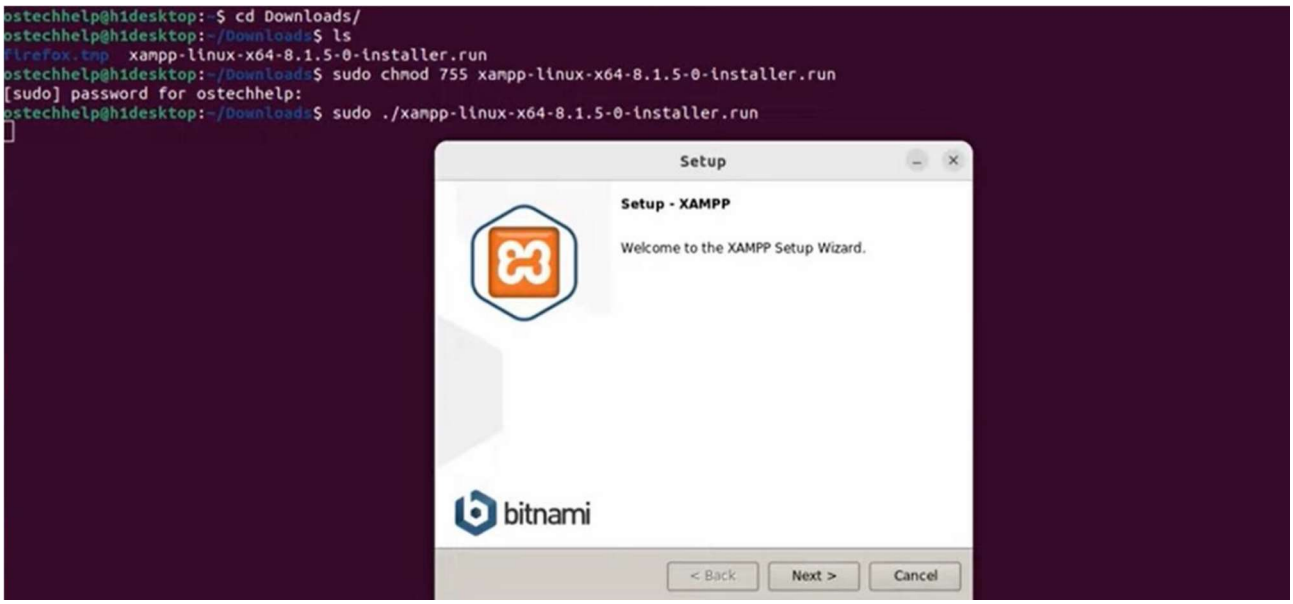
Name	Value/Description
async	A Boolean value indicating whether the request should be handled asynchronous or not. Default is true
contentType	The content type used when sending data to the server. Default is: "application/x-www-form-urlencoded"
data	Specifies data to be sent to the server
dataType	The data type expected of the server response.
error(*xhr,status,error*)	A function to run if the request fails.
success(*result,status,xhr*)	A function to be run when the request succeeds
type	Specifies the type of request. (GET or POST)
url	Specifies the URL to send the request to. Default is the current page

Steps to implement the program:

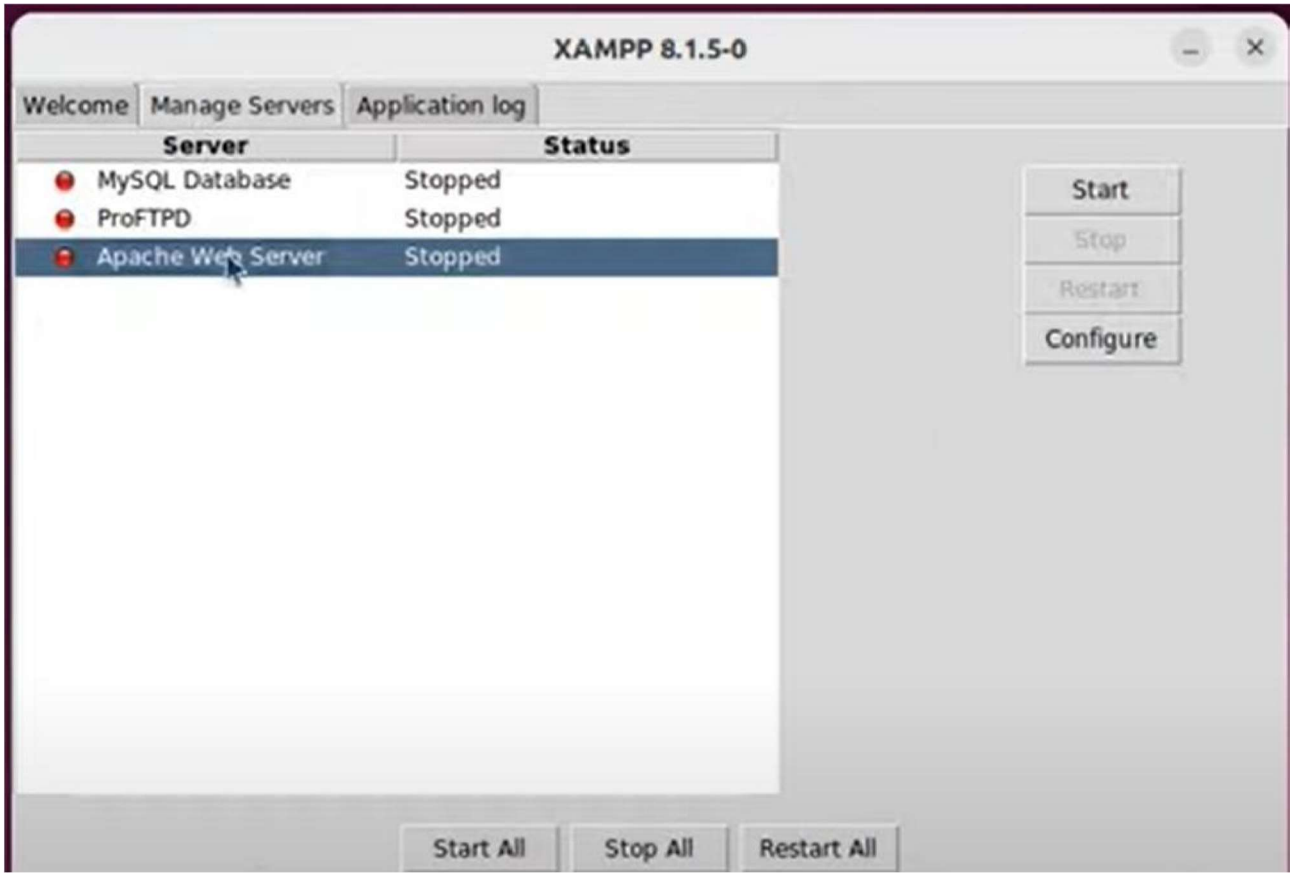
1. Install a web-server, here steps for installing XAMPP server on Ubuntu are mentioned Visit

   https://www.apachefriends.org/ and click on Ubuntu setup as highlighted below

2. Assuming that the setup is in Downloads, follow the steps ahead



Go on clicking next. After the setup is installed a following window will appear, click “Manage Servers” -> click on Apache Web Server -> click on Start



Once above setting is done, again go to terminal and use following commands

Here “mysite” is name of the folder and it will be launched in VSCode.

Suppose if you create “index.html” file in “mysite” folder, the on browser use the URL as:

localhost/mysite/index.html

Sample Code:

1. **Using XmlHttpRequest index.html**

```
<html>
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.6.3/jquery.min.js"></script>
<script src="https://code.jquery.com/jquery-3.6.3.js" integrity="sha256-
nQLuAZGRRcILA+6dMBOvcRh5Pe310sBpanc6+QBmyVM="
crossorigin="anonymous"></script>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-alpha1/dist/css/bootstrap.min.css"
rel="stylesheet">
<title>Assignment 1b Code</title>
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0alpha1/dist/js/bootstrap.bundle.min.js"></script>
<style>
form, h1{
width:30%;
margin: 5% auto;
}
input{
margin: 1%;
}
</style>
</head>
<body>
<h1>USER REGISTRATION</h1>
```

```
        <form action="" method="POST">
         <label>First Name</label>
            <input type="text" name="fullName" id="fullName" placeholder="Name Surname
Lastname" pattern="[A-Za-z ]*" class="form-control" required>
         <label>Email</label>
            <input type="email" name="userEmail" id="userEmail" class="form-control" required>
         <label>User Name</label>
            <input type="text" name="userName" id="userName" class="form-control" required>
         <label>Password</label>
            <input type="password" name="userPassword" id="userPassword" class="form-control"
required>
            <input type="button" onclick="submitForm()" name="submit" value="SUBMIT
DETAILS" id="submit" class="btn btn-primary w-100">
      </form>
      <!-- <div id="outputs"></div> -->
      <script>
  function submitForm(){
 var name = $('input[name=fullName]').val();
var email = $('input[name=userEmail]').val();
 var username = $('input[name=userName]').val();
var password = $('input[name=userPassword]').val();
  var data = 'name=' + name + '&email=' + email + '&username=' + username;
```

```
            xhr = new XMLHttpRequest();

  xhr.open('POST','output.php',true);

 xhr.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
xhr.onload = function(){

 //document.getElementById('outputs').innerHTML = xhr.responseText;

  var new_window = window.open(null, '','_blank');
new_window.document.write(xhr.responseText);

            }

  xhr.send(data);          }

      </script>

   </body>
</html>
```

Output.php

```
<h1>With XmlHttpRequest</h1>
<?php
          $name = $_POST['name'];
          $email = $_POST['email'];

         $username = $_POST['username'];

         echo "<div> Hi,".$name."</div>";

        echo "<div>Your email is: ".$email."</div>";
```

```
    echo "<div>Your username is: ".$username."</div>";

?>
```

---

2. Using $.ajax() **index.html**

```
<html>

  <head>

    <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.6.3/jquery.min.js"></script>
<script src="https://code.jquery.com/jquery-3.6.3.js" integrity="sha256-

nQLuAZGRRcILA+6dMBOvcRh5Pe310sBpanc6+QBmyVM="

crossorigin="anonymous"></script>

    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-alpha1/dist/css/bootstrap.min.css"
rel="stylesheet">

    <title>Assignment 1b Code</title>

    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0alpha1/dist/js/bootstrap.bundle.min.js"></script>

    <style>

 form, h1{
width:30%;
margin: 5% auto;

      }

  input{

  margin: 1%;       }

    </style>

    </head>

     <body>
```

```
<h1>USER REGISTRATION</h1>

  <form action="" method="POST">

   <label>First Name</label>

      <input type="text" name="fullName" id="fullName" placeholder="Name Surname
Lastname" pattern="[A-Za-z ]*" class="form-control" required>

   <label>Email</label>

      <input type="email" name="userEmail" id="userEmail" class="form-control" required>

   <label>User Name</label>
      <input type="text" name="userName" id="userName" class="form-control" required>

   <label>Password</label>

      <input type="password" name="userPassword" id="userPassword" class="form-control"
required>

      <input type="button" onclick="submitForm()" name="submit" value="SUBMIT
DETAILS" id="submit" class="btn btn-primary w-100">




</form>

<!-- <div id="outputs"></div> -->

<script>          function submitForm(){

 var name = $('input[name=fullName]').val();

var email = $('input[name=userEmail]').val();

 var username = $('input[name=userName]').val();
var password = $('input[name=userPassword]').val();

  var formData = 'name=' + name + '&email=' + email + '&username=' + username;
```

```
    $.ajax({
  type: "POST",
 url: "output.php",
   data: formData,
     }).done(function (response) {
                    //$("#outputs").html(response);

 var new_window = window.open(null, '','_blank');
new_window.document.write(response);

                 });
          }
        </script>
    </body>
</html>
```

Output.php

```
<h1>With Ajax</h1>
<?php
          $name = $_POST['name'];
          $email = $_POST['email'];
         $username = $_POST['username'];
         echo "<div> Hi,".$name."</div>";
         echo "<div>Your email is: ".$email."</div>";
```

```
    echo "<div>Your username is: ".$username."</div>";
?>
```

Conclusion: Thus, we have implemented a JavaScript Program to get the user registration data and using AJAX POST method we have listed data on new page.